

A Schema-Aware Verifier-First Agent for CAR-bench Track 1

BMAgent

CAR-bench IJCAI-ECAI 2026 Track 1 Submission

Abstract

CAR-bench evaluates whether LLM agents can operate as reliable in-car voice assistants under ambiguous requests, unavailable capabilities, and domain policies. We submit a Track 1 A2A-compatible tool-calling agent that keeps the core decision loop lightweight and moves reliability-critical logic into deterministic verification around each proposed tool call. The agent receives dynamic tool schemas from the evaluator, enriches them with compact tool-use guidance, asks a configurable LLM to propose the next assistant message or tool call, and then applies schema-aware checks, ID provenance constraints, conversation-state tracking, anti-loop guards, and structured-output normalization before returning an A2A response. This design targets the main failure modes observed during public validation: hallucinated tools or identifiers, repeated state-changing calls, premature final answers, failure to ask for clarification, and unsafe action after unreliable tool results. The submitted Docker image exposes all model, provider, API-base, deployment, service-tier, and reasoning settings through environment variables and reports A2A turn metrics where available.

1 Introduction

Real in-car assistants must complete user requests while respecting tool availability, vehicle state, user confirmation, and safety-related policies. CAR-bench formalizes this setting with Base, Hallucination, and Disambiguation tasks, an LLM-simulated user, 58 interconnected tools, and policy-driven evaluation [Kirmayr *et al.*, 2026]. A key property of the benchmark is consistency: hidden-set ranking is based on Pass³, where a task is counted as solved only if all three independent trials pass. This penalizes agents that occasionally succeed through lucky sampling but fail under small changes in user wording.

Our development objective was therefore not only to increase single-run success, but to reduce avoidable variance in the interaction loop. We evaluated planner/critic and workflow-skill variants, but the final Track 1 agent uses a single LLM as the action proposer and surrounds it with deter-

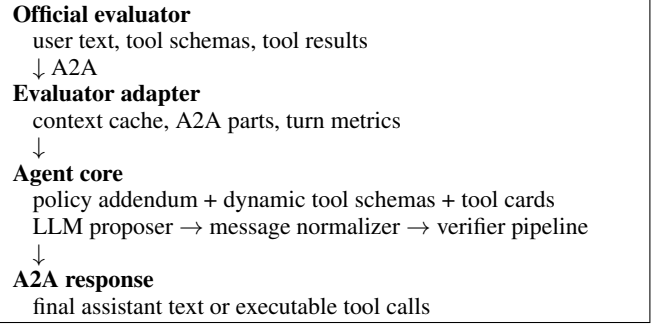


Figure 1: Track 1 architecture. Reliability is enforced after LLM proposal and before tool execution by the evaluator.

ministic scaffolding. This approach preserves the flexibility of tool calling while giving the system explicit checks for the parts that should not be left to model memory alone: schema validity, known identifier provenance, repeated actions, confirmation state, and capability limits.

2 System Overview

Figure 1 summarizes the submitted system. The official evaluator communicates with our agent through the A2A protocol. The adapter converts inbound A2A messages into an OpenAI-style message history, stores per-context state, and converts the agent core’s output back into A2A text or tool-call parts. The core then prepares dynamic tool definitions, calls the configured LLM, repairs provider-specific message formatting, runs deterministic verifiers, updates task state, and reports token/cost/latency metadata on final text responses.

2.1 A2A Adapter and Context Lifecycle

Each evaluator context is mapped to an agent state containing the full task-level conversation history. The state starts when the evaluator sends the initial system prompt and user observation, grows with each user message, assistant message, and tool result, and is dropped when the context is cancelled or evicted by the context cache. This state is short-lived working memory rather than a long-term vector memory. CAR-bench tasks are closed-world, task-specific interactions; the important information is not external knowledge, but which tools

have already been called, what results they returned, whether the user confirmed an action, and whether the last assistant step made progress.

The adapter also stores the latest dynamic tool schema for the context. This is important because the official evaluator, not the participant, owns the tool definitions. Our agent never assumes that a tool exists merely because it appeared in public development data; the current evaluator-supplied schema is the source of truth for executable tools.

2.2 Model and Provider Abstraction

The submitted image is model configurable. The scenario exposes environment variables for the model name, provider route, API base URL, deployment name, service tier, reasoning effort, and optional thinking controls. We use LiteLLM-compatible completion calls for Track 1 routing, with aliases for OpenAI, Anthropic/Claude, Gemini, Azure, and DeepSeek-style providers. The submitted default route is Claude Opus 4.6 through the direct Anthropic API, while public validation was also run with DeepSeek V4 Flash. Temperature is set to zero by default to reduce trial-to-trial variance.

3 Verifier-First Tool Calling

The central design choice is to treat the LLM as a candidate-action generator, not as the only reliability mechanism. After each model call, the agent normalizes the message and applies a fixed verifier pipeline. If a tool call is invalid or unsafe, the verifier either removes the problematic call, converts the step into a clarification or refusal, or returns a concise completion message when the action has already been completed.

3.1 Schema-Aware Validation

The evaluator sends tool schemas at runtime. The agent checks generated tool calls against the currently available tool names and argument schemas. Unknown tools, missing required arguments, and unsupported top-level fields are blocked or repaired before execution. This reduces tool hallucination: the model may propose a plausible tool name or parameter, but the agent refuses to forward it unless the evaluator schema allows it.

We also attach compact tool cards to tool descriptions. A tool card contains a natural-language description plus `use_when`, `not_when`, `requires`, and `after` fields. These cards guide the LLM before generation, while the verifier remains the hard constraint after generation. The separation matters: prompting helps select the right tool, but only the deterministic schema check can guarantee that impossible tools are not executed.

3.2 ID Provenance Constraints

Many CAR-bench tools operate on identifiers such as `route_id`, `contact_id`, `poi_id`, and route segment IDs. A common failure mode is to pass a human-readable name, an invented ID, or a stale ID into a later tool call. The agent derives allowed ID sets from previous tool results in the current conversation. For example, contact IDs must come from contact lookup results before contact details can be requested,

and navigation route IDs must come from route tools before route editing or navigation setup. For multi-stop navigation, the verifier also checks that adjacent route segments connect in travel order.

This mechanism reduces both hallucination and drift. It prevents the agent from “remembering” an ID that was never returned, and it catches cases where the model mixes the route leading to a waypoint with the route leaving that waypoint.

3.3 Conversation-State Tracking

The full message history is sent to the LLM, but the verifier also derives structured state from it. This derived working memory includes successful tool signatures, recent state-changing actions, known contact/route/POI records, successful email sends, user confirmation text, STOP signals, unreliable tool-result markers, and repeated search signatures. The derived state is not a separate long prompt by default; it is an execution-time control layer. This avoids duplicating large portions of the context while still letting deterministic code enforce state-sensitive constraints.

3.4 Anti-Loop and Progress Guards

Repeated tool calls waste budget and often reduce Pass³. The agent therefore removes exact duplicate tool calls emitted in the same assistant turn and blocks repeated completed actions in the recent history unless the latest user explicitly asks to repeat them. It also blocks repeated POI searches with the same route, category, and kilometer after a non-empty successful result. STOP handling is another guard: if the user simulator emits a stop signal, the agent must not start new work.

The guards are intentionally narrow. Repetition is allowed when the parameters differ, when an earlier attempt failed, when a fresh user instruction asks for another attempt, or when a later tool requires a different upstream lookup. This keeps the anti-loop mechanism from preventing legitimate multi-step workflows.

4 Handling Ambiguity and Missing Capabilities

CAR-bench rewards agents that ask for clarification when the user request omits necessary information and that admit limits when required capabilities or reliable facts are unavailable. Our system combines prompting and verification for these cases. The policy addendum instructs the model to ask targeted questions instead of guessing user preferences or hidden entities. The verifier then catches high-risk mistakes: action after an unreliable result, claims about unavailable capabilities, route editing without required route data, and state changes after a successful but already completed workflow.

For ambiguous user requests, the agent is encouraged to inspect current state and user preferences through tools when such tools exist, then ask only for the missing piece. For hallucination-style tasks, the verifier prevents fallback behavior that would fabricate a result from text alone. When the evaluator tool reports missing or unreliable data, subsequent dependent actions are blocked and converted into a transparent user-facing explanation.

Public split	Tasks	Pass ³	Pass@3
Base	50	0.62	0.82
Disambiguation	25	0.64	0.96
Hallucination	50	0.50	0.78

Table 1: Representative local public validation with three trials per task. Reported numbers are from development traces and are not hidden-set results.

5 Structured Output and Observability

Although the submitted agent itself returns A2A messages and tool calls, local public validation also depends on LLM-based user simulation and policy evaluation. We added a local structured-output normalizer for development runs: raw model output is stripped of markdown fences, parsed as JSON, normalized into the expected Pydantic schema, and retried only a bounded number of times. This prevents a model’s malformed simulator output from obscuring agent failures during local debugging. The official evaluator remains external and is not modified by the submitted image.

Every local run writes `records.jsonl`, `summary.json`, and per-task logs. The records include rewards, task IDs, errors, and interaction traces; the summary reports average reward, Pass³, Pass@3, and failed task IDs. This logging was essential for finding recurrent failure modes such as duplicate sends, stale route IDs, and premature final responses. The submitted A2A server also reports public turn metrics where available: prompt tokens, completion tokens, thinking tokens, cost delta, model name, call count, and LLM latency.

6 Public Validation

Table 1 reports representative public-set validation traces from development, collected with DeepSeek V4 Flash as the agent/user/evaluator model in the local runner. These are not hidden-set scores and should be interpreted as local diagnostics; official ranking is determined only by the organizer-run hidden evaluation. The values nevertheless show why Pass³ was the optimization target: Pass@3 can be high while Pass³ remains much lower, indicating unstable tasks that require verifier and state-tracking work rather than more attempts.

7 Ablations and Lessons Learned

Several development variants were useful but were not kept as the final default. A planner/critic scaffold produced more explicit reasoning traces, but it added sequential LLM calls and sometimes re-planned after a partially completed workflow, increasing latency and tool-loop risk. Broad workflow-skill injection helped a few repeated task patterns, but it over-constrained unrelated tasks and reduced cross-category stability. The strongest trade-off came from narrower deterministic verifiers: they do not try to solve the task by themselves, but they reliably reject impossible tools, invented IDs, stale route segments, repeated completed actions, and unsafe actions after unreliable tool results. This lesson shaped the final design: use prompting for intent selection and deterministic checks for invariant enforcement.

Qualitatively, the most common residual failures are complex navigation edits, charging-route composition, and tasks where the simulator’s paraphrase hides the exact missing preference. The final agent therefore keeps optional workflow skills disabled in the official scenario and relies on narrower verifier rules.

8 Submission and Compliance

The agent is packaged as a public digest-pinned GHCR image and configured by `submission/scenario.toml`. The scenario uses the official evaluator image, sets `task_split = "hidden"`, uses `-1` task counts for all public categories, and contains only environment variable names rather than secret values. The submitted image does not inspect hidden task files, evaluator internals, answer keys, or private evaluator state. All model and provider choices required by the agent are environment-configurable, including API bases and reasoning-effort selectors.

Compute and rate-limit assumptions. The official scenario uses one primary LLM call per agent turn. The verifier pipeline is deterministic Python code and does not add extra model calls in the submitted configuration. Temperature is zero by default, request timeouts are configurable, and token usage is reported through A2A turn metrics when provider usage metadata is available. This design assumes standard hosted API rate limits for the selected Track 1 provider; if organizers switch providers through environment variables, the same single-call-per-turn scaffold and verifier pipeline are preserved.

9 Conclusion

Our Track 1 submission is a lightweight, provider-configurable, A2A-compatible tool-calling agent optimized for reliability rather than elaborate planning. The main contribution is a verifier-first execution scaffold: dynamic schema adaptation, ID provenance constraints, task-level working memory, anti-loop guards, structured local validation, and full observability. This architecture directly targets CAR-bench’s deployment-oriented failure modes and keeps latency and cost manageable while improving consistency across repeated trials.

References

- [Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth Andre. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, San Diego, California, United States, July 2026. Association for Computational Linguistics.